矩形检测

矩形检测
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- 矩形检测函数
  - find_rects()

例程实验效果简介

本节例程代码位于：【源码汇总 / 04.Detecting / 02.find_rects.py】

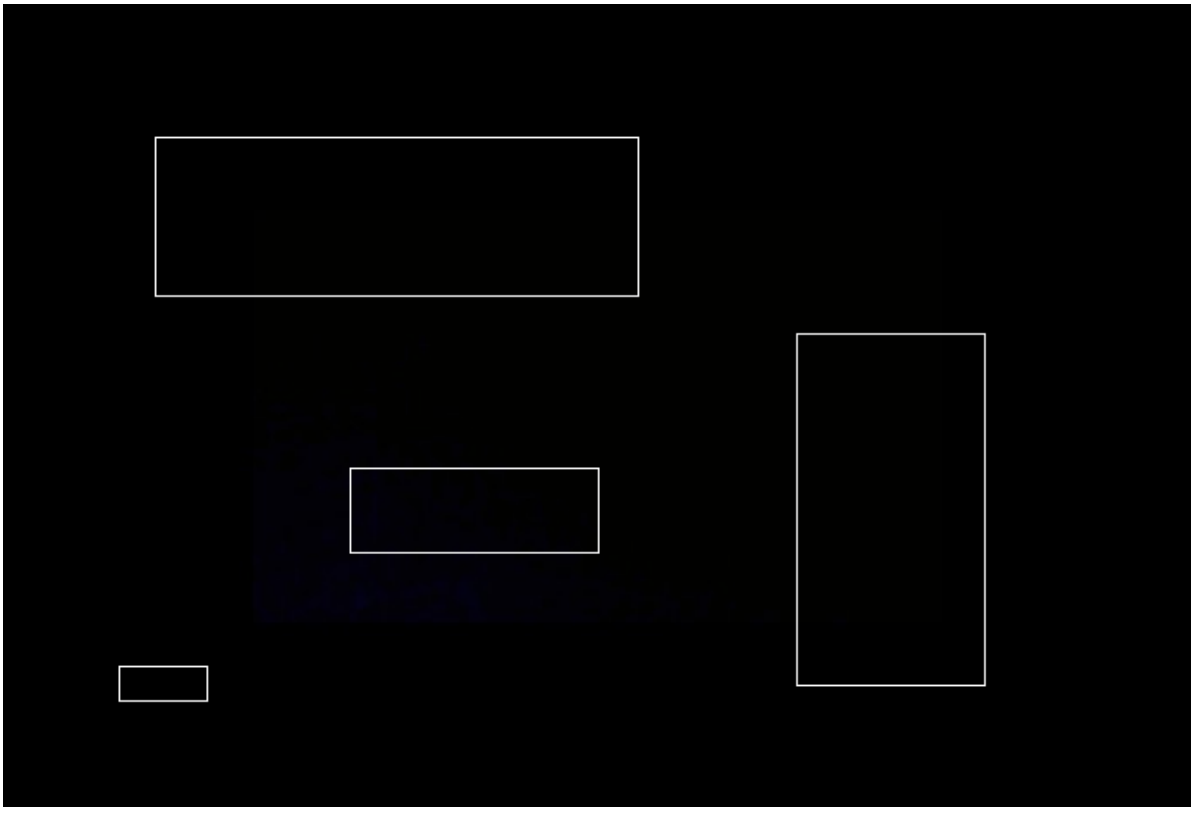
我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮，

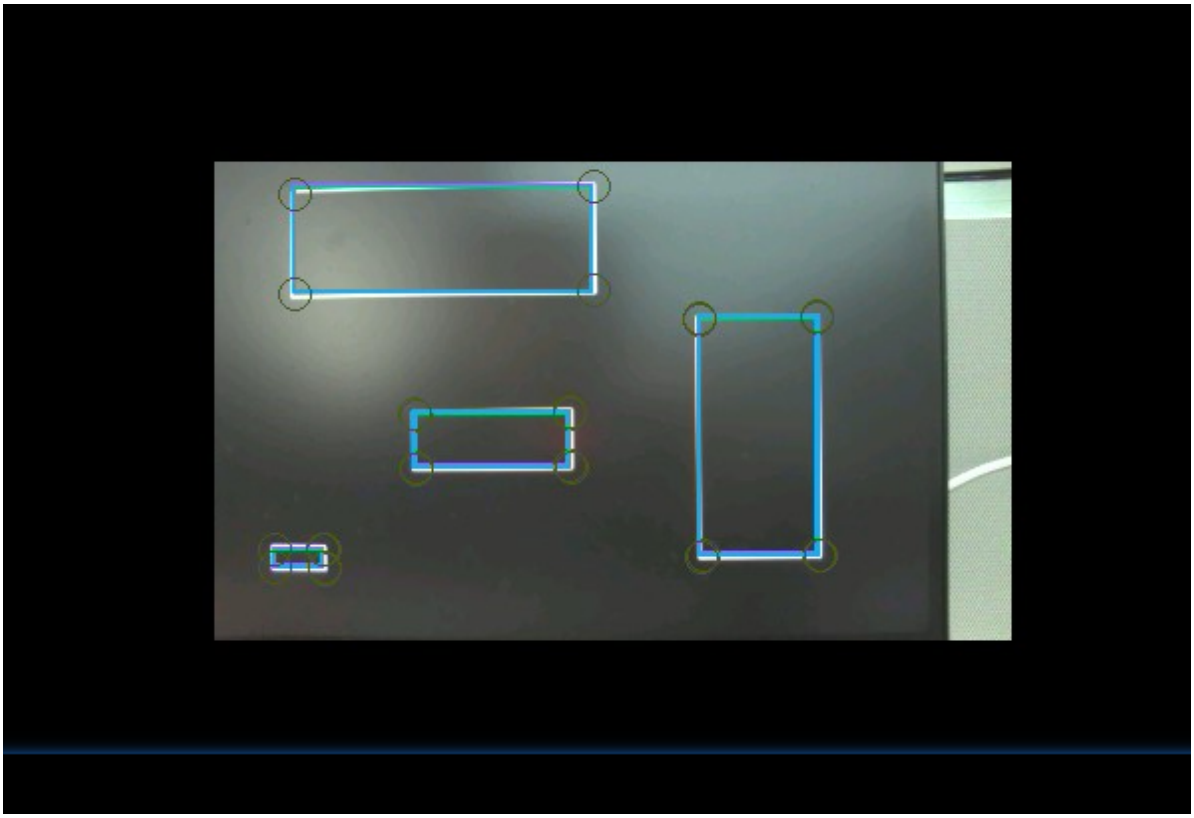
将K230的摄像头对准矩形框

可以看到屏幕上会标记出画面中的 **矩形**（如果没有屏幕就看帧缓冲区）

【原图】



【K230识别】



代码讲解

本节我们要用的的外设主要是摄像头模块

这一节的代码和上节检测线段在逻辑上是一致的，区别在于使用了find_rects方法去检测矩形边框

矩形检测由 K230中的 find_rects() 方法实现，该方法属于image模块

完整代码

```
# 导入必要的模块 Import required modules
import time, os, sys
from media.sensor import *
from media.display import *
from media.media import *

# 图像分辨率设置 Image resolution settings
PICTURE_WIDTH = 400
PICTURE_HEIGHT = 240

# 摄像头配置 Camera configuration
sensor = None

# 显示模式选择 Display mode selection
# 可选: "VIRT", "LCD"
# Options: "VIRT"(Virtual Display), "LCD"
DISPLAY_MODE = "LCD"

# 根据显示模式设置分辨率 Set resolution based on display mode
if DISPLAY_MODE == "VIRT":
    DISPLAY_WIDTH = ALIGN_UP(1920, 16)
    DISPLAY_HEIGHT = 1080
```

```
elif DISPLAY_MODE == "LCD":
    DISPLAY_WIDTH = 800
    DISPLAY_HEIGHT = 480
else:
    raise ValueError("Unknown DISPLAY_MODE, please select 'VIRT', 'LCD'")

# 创建时钟对象用于FPS计算 Create clock object for FPS calculation
clock = time.clock()

try:
    # 初始化摄像头 Initialize camera
    sensor = Sensor(id=2)
    sensor.reset()

    # 设置图像分辨率和格式 Set image resolution and format
    sensor.set_framesize(width=PICTURE_WIDTH, height=PICTURE_HEIGHT,
chn=CAM_CHN_ID_0)
    sensor.set_pixformat(Sensor.RGB565, chn=CAM_CHN_ID_0)

    # 初始化显示器 Initialize display
    if DISPLAY_MODE == "VIRT":
        Display.init(Display.VIRT, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
fps=60)
    elif DISPLAY_MODE == "LCD":
        Display.init(Display.ST7701, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
to_ide=True)

    # 初始化媒体管理器 Initialize media manager
    MediaManager.init()
    sensor.run()

    while True:
        os.exitpoint()
        clock.tick() # 开始计时 Start timing

        # 捕获图像 Capture image
        img = sensor.snapshot(chn=CAM_CHN_ID_0)

        print("【矩形信息 Line Statistics Start】")


        for r in img.find_rects(threshold = 8000):
             img.draw_rectangle(r.rect(), color = (40, 167, 225),thickness=2)
             for p in r.corners(): img.draw_circle(p[0], p[1], 8, color = (78,
90, 34))
             print(r)
        print("【==============================】")

        # 显示FPS Display FPS
        print(f"FPS: {clock.fps()}")

        # 居中显示图像 Display image centered
        x = int((DISPLAY_WIDTH - PICTURE_WIDTH) / 2)
        y = int((DISPLAY_HEIGHT - PICTURE_HEIGHT) / 2)
        Display.show_image(img, x=x, y=y)

except KeyboardInterrupt as e:
    print("User Stop: ", e)
```

```
except BaseException as e:
    print(f"Exception: {e}")
finally:
    # 清理资源 Cleanup resources
    if isinstance(sensor, Sensor):
        sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
    MediaManager.deinit()
```

代码结构

基础设置：

- 设置图像分辨率(400x240)
- 支持两种显示模式：虚拟显示器(VIRT, 1920x1080)和LCD显示器(800x480)

主要功能流程：

- 初始化摄像头，设置分辨率和RGB565颜色格式
- 初始化显示器
- 进入主循环：
  - 实时捕获图像
  - 检测图像中的矩形
  - 在检测到的矩形上绘制边框(蓝色)和角点(深绿色)
  - 显示FPS(帧率)
  - 将图像居中显示在屏幕上

异常处理：

- 捕获键盘中断和其他异常
- 程序结束时清理资源：停止摄像头、关闭显示器、释放媒体资源

矩形检测函数

find_rects()

```
image.find_rects([roi=Auto, threshold=10000])
```

使用用于查找AprilTag的相同的quad detection算法来查找图像中的矩形。 最适用与背景形成鲜明对比的矩形。AprilTag的quad detection可以处理任意缩放/旋转/剪切的矩形。 返回一个 image.rect 对象的列表。

roi 是一个用以复制的矩形的感兴趣区域(x, y, w, h)。如果未指定， ROI即图像矩形。操作范围仅限于 roi 区域内的像素。

边界大小（通过在矩形边缘上的所有像素上滑动索贝尔算子并相加该值）小于 threshold 的矩形会从返回列表中过滤出来。 threshold 的正确值取决于您的应用程序/场景。

不支持压缩图像和bayer图像。